

TUTORIAL SESSIONS



5210COMP:

Software Engineering for Games

Module Leader: Dr Chris Carter

Tutorial 05: SFML Tutorial #03 Animated Sprite Game Objects

Important: Finish all tasks in the previous tutorial before starting these tasks

Tutorial Overview

This tutorial will help you to develop a `GameObject` that contains an animated sprite. In this tutorial you will:

- The **First Task** create a new `GameObject` that contains an animated sprite.
- The **Second Task** is to add an instance of the `GameObjectWithAnimatedSprite` to the `GameState`.
- The **Third Task** is to tell the animation to animate!
- The **Fourth Task** is to manipulate the `GameObjectWithAnimatedSprite` using translation, rotation, and scaling functions.
- Additional tasks are for you to work on at home.

Pre-Requirement Task:

In Tutorial 02 and 03, we developed the base Sprite and Text classes.

In Tutorial 04, we set up our Perforce Repositories for the Groups and Committed the initial version of the game.

If you have not done this for your group, this should be the first thing that you do today.

You should catch up on Today's tutorial in your independent study time if you need to do Tutorials 03 and 04 first.

Learning Task : Programming Support on Tutorials

If you have been struggling with any of the concepts that we have been covering in the development tutorials – it is likely that you have a few gaps in your C++ knowledge.

This is a module where you cannot bury your heads in the sand! We need to address these gaps in order to progress – we cannot write game code if we don't understand some of the big features of C++ OO.

Therefore, today's tutorial also includes some Sample Code across Four different categories – this code is a companion to the materials relating to Lecture 05 – it is important that you study and experiment with this code to gain a better appreciation of some core OO Programming concepts.

This code does not use the Engine, it is designed to show you how classes are built from Scratch and is specifically designed to illustrate concepts.

Even if you have a good understanding of what we have covered so far, it is advised that you have a look at the code, even if you are confident with the concepts – it will always make good reference for the Syntax of C++ and some of the common mistakes we can make.

Try and attempt today's tutorial first – if you get stuck, come back and do this task and have a good look over the source code.

Each code sample is provided as a Visual Studio Zip file – just extract and open in Visual Studio 2022 – each sample is written as a small, concise demo which has a Source.cpp file containing the main functions, plus some small OO class files.

- Concept 1: Formal Syntax of Specifying Classes
- Concept 2: Functions and Passing Data to Functions
- Concept 3: Classes within Classes
- Concept 4: Association, Composition and Aggregation, along with Dynamic vs standard instance data.

It is recommended that you use this code as reference, as you progress through the tutorials to give you some tips about what you should be doing and the syntax and semantics of this implementation in C++.

Task 1: Create a GameObject that has an Animated Sprite

The **AnimatedSprite** class in the SEG Engine that I have written in this engine is just a wrapper for the SFML sprite class that you used in the 2D Computer Graphics module that you studied at level 4 – it functions very similarly to that of the alternative framework version that is on Canvas – that version focuses on the use of data structures – this class uses the same structures, but is designed to be data driven. You can look at that example if you wish before starting today's tutorial.

This wrapper class is used to integrate the SFML sprite class into the SEG Engine and provide easy access to the AssetManager and RenderManager that take care of loading textures and rendering the sprite to the screen.

Step 1: Create a new class that inherits from GameObject

Create a new class that inherits from **GameObject** and call it **GameObjectWithAnimatedSprite**. This should go in the same location as your other objects. It is suggested that you use the Class wizard in Visual Studio to help you do this and then move the .h and .cpp files generated to the correct folder (using Show all files).

Step 2: Add the required functions

As you did in the previous tutorials, when creating the **GameObjectWithAnimatedSprite**, add in all of the functions required for a class that inherits from **GameObject**.

Step 3: Add an AnimatedSprite member

Add an instance of the **AnimatedSprite** class (not a pointer) as a **private (or protected)** class member called **m_AnimatedSprite**; you will need to include **AnimatedSprite.h** as usual.

SUGGESTED APPROACH: Try using ReSharper to do this, instead of the manually typed approach that we have taken so far – if you have not done so already, have a look on today's Canvas site for details of obtaining ReSharper for C++.

If you declare the member variable first, you can right-click on the **AnimatedSprite** data type and should find a ReSharper menu item which will help you!

Step 4: Initialise m_AnimatedSprite (Most difficult task here)

The **AnimatedSprite** class has two methods of adding animations.

The first method is by calling the **AddAnimation()** Member function, which will load a regular image file where you must supply the frame rectangles (rects) and the pivot point for movement / rotation / scaling (origins).

Call this function, passing in the following information for your parameters (*you will need to work out the order of the parameters based on the text of the function*):

- spritesheet file : "sprites/spritesheet_test.png"
- sprite frame width: 256
- sprite frame height: 256
- number of sprite frames in the animation: 6
- frame rate: 10
- The animation should loop.

Hints: You will have to open up *AnimatedSprite.h* and look at the interface (public functions) of the class to see what data types it is expecting. Open the *spritesheet_test* from the Image folder and see if you can work out the 2D position coordinates for the top-left of each frame and how big each frame is.

You also need to work out the origin of the sprite – this origin should be the centre point of each frame (so you know the width and height – what is the center?)

You will also need to be able to create a Dynamic List in C++ (Vector) for each of the Frame Rectangles and the Origin Positions. You should use either the `push_back` or `emplace_back` function to help you populate these lists. Based on the information on the image.

This should help you:

- <https://www.learncpp.com/cpp-tutorial/an-introduction-to-stdvector/>
- <https://www.learncpp.com/cpp-tutorial/stl-containers-overview/>
- <https://cplusplus.com/forum/beginner/158723/>

These lists must be passed into the function as parameters so each will need a named variable.

Step 5: Making sure you set the Initial Animation:

After you have called `AddAnimation()` you must set the current animation using the `SetCurrentAnimation()` function.

Step 6: Fill in the `GetPosition` and `SetPosition` functions

As our animated sprite member has a position (since it inherits from SFML's transformable class), we can return this in the `GetPosition` functions.

Inside the `GetPosition` function set the x and y reference parameters' values by calling `getPosition().x` and `getPosition().y` on the `m_AnimatedSprite` member.

Have a look at your code for `Sprite` and `Text` and see if you can apply the same concepts to complete all the `GetPosition` and `SetPosition` Variants.

Task 2: Add an instance of the `GameObjectWithAnimatedSprite` to the `GameStateTest` `GameState`.

If we want to see our new class in action, we need to create an instance of it inside a game state.

Step 1: Create an instance of the `GameObjectWithAnimatedSprite` class

Open the `GameStateTest.cpp` file and locate the `Initialise()` function.
Create a local temporary variable that is a pointer to a `GameObjectWithAnimatedSprite` class and initialize it using the **new** keyword – this is the same as we did with the other game objects.

Step 2: Initialise the `GameObjectWithAnimatedSprite`

Call the **Initialise** function on the pointer that you created in the previous step.

Step 3: Add the `GameObjectWithAnimatedSprite` to the `GameObject` list

Call `push_back()` on the `m_GameObjects` member variable, passing in the pointer to the `GameObjectWithAnimatedSprite` instance variable that you have created in the previous steps.

Task 3: Animate!

You will notice that your animation isn't doing anything. That's because we need to tell it to animate. Because we added the Game Object to the list of game objects, we know that its update method is being called once per frame, so we can leverage this to provide animation for our Animated Sprite Game Object

Step 1: Call Animate from the GameObject's Update.

In the update function for the GameObjectWithAnimatedSprite call the Animate() function on the m_AnimatedSprite member object.

Task 4: Use translation, rotation and scale to manipulate the GameObjectWithAnimatedSprite

As the AnimatedSprite class is just a wrapper for the SFML sprite class, we have access to all of the SFML sprite functions that you used in previous modules to transform our object (via the Transformable Interface/Data Class of SFML).

Let's use these functions to manipulate the sprite on the screen.

Every GameObject that is registered to a Game state will automatically have the Update() function called every frame until it is destroyed.

Step 1: Move (translate) the sprite

Locate the Update() function of the GameObjectWithAnimatedSprite class. Experiment with the move() and functions on the sprite member object, m_AnimatedSprite, to move the sprite around the screen.

Use the SetPosition functions of GameObjectWithAnimatedSprite in the GameStateTest class to change the starting position of the animated sprite.

Use the same concepts from Tutorials 2 and 3 to help you.

This time, the reference for this should be the engine's AnimatedSprite class to see what functions you have access to, alongside the SFML documentation.

Step 2: Rotate the sprite

Within the Update() function of the GameObjectWithAnimatedSprite class. Experiment with the rotate() and setRotation() functions on the sprite member object, m_AnimatedSprite, to rotate the animated sprite as it moves around the screen.

Step 3: Scale the sprite

Within the Update() function of the GameObjectWithAnimatedSprite class. Experiment with the scale() and setScale() functions on the animated sprite member object, m_AnimatedSprite, to scale the sprite as it moves and rotates around the screen.

Additional Task: Make the GameObjectWithAnimatedSprite generic

You don't want to write a new class every time you want to put an animated sprite on the screen, particularly if it is only differing by a few properties such as the image or size etc.

Make the GameObjectWithAnimatedSprite class generic so that you can make specific customizations to the sprite.

Suggested changes:

- Position
- Colour
- Scale
- Rotation
- **Any more you can think of proving to be useful!**

Unlike in the alternative framework version, where we gave direct access to the SFML class, to demonstrate structure – we are adopting an OO Approach known as a Wrapper class – typically implemented using a Design Pattern called the Adapter (more on this later) – this typically relies on a composition relationship but can be used with Aggregation as well.

Have a look at the following to help you understand this pattern:

<https://www.geeksforgeeks.org/adapter-pattern/>